

TENSORFLOW LITE MICRO: EMBEDDED MACHINE LEARNING ON TINYML SYSTEMS

Robert David¹ Jared Duke¹ Advait Jain¹ Vijay Janapa Reddi^{1,2}
 Nat Jeffries¹ Jian Li¹ Nick Kreeger¹ Ian Nappier¹ Meghna Natraj¹
 Shlomi Regev¹ Rocky Rhodes¹ Tiezheng Wang¹ Pete Warden¹

ABSTRACT

Deep learning inference on embedded devices is a burgeoning field with myriad applications because tiny embedded devices are omnipresent. But we must overcome major challenges before we can benefit from this opportunity. Embedded processors are severely resource constrained. Their nearest mobile counterparts exhibit at least a 100—1,000x difference in compute capability, memory availability, and power consumption. As a result, the machine-learning (ML) models and associated ML inference framework must not only execute efficiently but also operate in a few kilobytes of memory. Also, the embedded devices’ ecosystem is heavily fragmented. To maximize efficiency, system vendors often omit many features that commonly appear in mainstream systems, including dynamic memory allocation and virtual memory, that allow for cross-platform interoperability. The hardware comes in many flavors (e.g., instruction-set architecture and FPU support, or lack thereof). We introduce TensorFlow Lite Micro (TF Micro), an open-source ML inference framework for running deep-learning models on embedded systems. TF Micro tackles the efficiency requirements imposed by embedded-system resource constraints and the fragmentation challenges that make cross-platform interoperability nearly impossible. The framework adopts a unique interpreter-based approach that provides flexibility while overcoming these challenges. This paper explains the design decisions behind TF Micro and describes its implementation details. Also, we present an evaluation to demonstrate its low resource requirement and minimal run-time performance overhead.

1 INTRODUCTION

Tiny machine learning (TinyML) is a burgeoning field at the intersection of embedded systems and machine learning. The world has over 250 billion microcontrollers (IC Insights, 2020), with strong growth projected over coming years. As such, a new range of embedded applications are emerging for neural networks. Because these models are extremely small (few hundred KBs), running on microcontrollers or DSP-based embedded subsystems, they can operate continuously with minimal impact on device battery life.

The most well-known and widely deployed example of this new TinyML technology is keyword spotting, also called hotword or wakeword detection (Chen et al., 2014; Gruenstein et al., 2017; Zhang et al., 2017). Amazon, Apple, Google, and others use tiny neural networks on billions of devices to run always-on inferences for keyword detection—and this is far from the only TinyML application. Low-latency analysis and modeling of sensor signals from microphones, low-power image sensors, accelerometers, gyros,

PPG optical sensors, and other devices enable consumer and industrial applications, including predictive maintenance (Goebel et al., 2020; Susto et al., 2014), acoustic-anomaly detection (Koizumi et al., 2019), visual object detection (Chowdhery et al., 2019), and human-activity recognition (Chavarriaga et al., 2013; Zhang & Sawchuk, 2012).

Unlocking machine learning’s potential in embedded devices requires overcoming two crucial challenges. First and foremost, embedded systems have no unified TinyML framework. When engineers have deployed neural networks to such systems, they have built one-off frameworks that require manual optimization for each hardware platform. These frameworks have tended to be narrowly focused, lacking features to support multiple applications and lacking portability across a wide range of hardware. The developer experience has therefore been painful, requiring hand optimization of models to run on a specific device. And altering these models to run on another device necessitated manual porting and repeated optimization effort. A second-order effect of this situation is that the slow pace and high cost of training and deploying models to embedded hardware prevents developers from easily justifying the investment required to build new features.

¹Google ²Harvard University. Correspondence to: Pete Warden <petewarden@google.com>, Vijay Janapa Reddi (contributions made as a Google Visiting Researcher) <vj@eecs.harvard.edu>.

Another challenge limiting TinyML is that hardware vendors have related but separate needs. Without a generic TinyML framework, evaluating hardware performance in a neutral, vendor-agnostic manner has been difficult. Frameworks are tied to specific devices, and it is hard to determine the source of improvements because they can come from hardware, software, or the complete vertically integrated solution.

The lack of a proper framework has been a barrier to accelerating TinyML adoption and application in products. Beyond deploying a model to an embedded target, the framework must also have a means of training a model on a higher-compute platform. TinyML must exploit a broad ecosystem of tools for ML, as well for orchestrating and debugging models, which are beneficial for production devices.

Prior efforts have attempted to bridge this gap, and we discuss some of them later (Section 6). Briefly, we can distill the general issues facing many of the frameworks into the following:

- Inability to easily and portably deploy models across multiple embedded hardware architectures
- Lack of optimizations that take advantage of the underlying hardware without requiring framework developers to make platform-specific efforts
- Lack of productivity tools that connect training pipelines to deployment platforms and tools
- Incomplete infrastructure for compression, quantization, model invocation, and execution
- Minimal support features for performance profiling, debugging, orchestration, and so on
- No standard benchmarks that allow hardware vendors to quantify their chip's performance in a fair and reproducible manner
- Lack of testing in real-world applications.

To address these issues, we introduce TensorFlow Lite Micro (TF Micro), which mitigates the slow pace and high cost of training and deploying models to embedded hardware by emphasizing portability and flexibility. TF Micro makes it easy to get TinyML applications running across architectures, and it allows hardware vendors to incrementally optimize kernels for their devices. TF Micro gives vendors a neutral platform to prove their hardware's performance in TinyML applications. It offers these benefits:

- Our interpreter-based approach is portable, flexible, and easily adapted to new applications and features
- We minimize the use of external dependencies and library requirements to be hardware agnostic

- We enable hardware vendors to provide platform-specific optimizations on a per-kernel basis without writing target-specific compilers
- We allow hardware vendors to easily integrate their kernel optimizations to ensure performance in production and comparative hardware benchmarking
- Our model-architecture framework is open to a wide machine-learning ecosystem and the TensorFlow Lite model conversion and optimization infrastructure
- We provide benchmarks that are being adopted by industry-leading benchmark bodies like MLPerf
- Our framework supports popular, well-maintained Google applications that are in production.

This paper makes several contributions: First, we clearly lay out the challenges to developing a machine-learning framework for embedded devices that supports the fragmented embedded ecosystem. Second, we provide design and implementation details for a system specifically created to cope with these challenges. And third, we demonstrate that an interpreter-based approach, which is traditionally viewed as a low-performance alternative to compilation, is in fact highly suitable for the embedded domain—specifically, for machine learning. Because machine-learning performance is largely dictated by linear-algebra computations, the interpreter design imposes minimal run-time overhead.

2 TECHNICAL CHALLENGES

Many issues make developing a machine-learning framework for embedded systems particularly difficult. In this section, we summarize the main ones.

2.1 Missing Features

Embedded platforms are defined by their tight limitations. Therefore, many advances from the past few decades that have made software development faster and easier are unavailable to these platforms because the resource tradeoffs are too expensive. Examples include dynamic memory management, virtual memory, an operating system, a standard instruction set, a file system, floating-point hardware, and other tools that seem fundamental to modern programmers (Kumar et al., 2017). Though some platforms provide a subset of these features, a framework targeting widespread adoption in this market must avoid relying on them.

2.2 Fragmented Market and Ecosystem

Many embedded-system uses only require fixed software developed alongside the hardware, usually by an affiliated team. The lack of applications capable of running

on the platform is therefore much less important than it is for general-purpose computing. Moreover, backward instruction-set-architecture (ISA) compatibility with older software matters less than in mainstream systems because everything that runs on an embedded system is probably compiled from source code anyway. Thus, the hardware can aggressively diversify to meet power requirements, whereas even the latest x86 processor can still run instructions that are nearly three decades old (Intel, 2013).

These differences mean the pressure to converge on one or two dominant platforms or ISAs is much weaker in the embedded space, leading to fragmentation. Many ISAs have thriving ecosystems, and the benefits they bring to particular applications outweigh developers' cost of switching. Companies even allow developers to add their own ISA extensions (ARM, 2019; Waterman & Asanovic, 2019).

Matching the wide variety of embedded architectures are the numerous tool chains and integrated development environments (IDEs) that support them. Many of these systems are only available through a commercial license with the hardware manufacturer, and in cases where a customer has requested specialized instructions, they may be inaccessible to everyone. These arrangements have no open-source ecosystem, leading to device fragmentation that prevents a lone development team from producing software that runs well on many different embedded platforms.

2.3 Resource Constraints

People who build embedded devices do so because a more general-purpose computing platform exceeds their design limits. The biggest drivers are cost, with a microcontroller typically selling for less than a few dollars (IC Insights, 2020); power consumption, as embedded devices may require just a few milliwatts of power, whereas mobile and desktop CPUs require watts; and form factor, since capable microcontrollers are smaller than a grain of rice (Wu et al., 2018).

To meet their needs, hardware designers trade off capabilities. A common characteristic of an embedded system is its low memory capacity. At one end of the spectrum, a big embedded system has a few megabytes of flash ROM and at most a megabyte of SRAM. At the other end, a small embedded system has just a few hundred kilobytes or fewer, often split between ROM and RAM (Zhang et al., 2017).

These constraints mean both working memory and permanent storage are much smaller than most software written for general-purpose platforms would assume. In particular, the size of the compiled code in storage requires minimization.

Most software written for general-purpose platforms contains code that often goes uncalled on a given device. The reason is that choosing the code path at run time is a bet-

ter use of engineering resources than shipping more-highly custom executables. This run-time flexibility is hard to justify when code size is a concern and the potential uses are fewer. As a result, developers often must break through the a library's abstraction if they want to make modifications to suit their target hardware.

2.4 Ongoing Changes to Deep Learning

Machine learning remains in its infancy despite its break-neck pace. Researchers are still experimenting with new operations and network architectures to glean better predictions from their models. Their success in improving results leads product designers to demand these enhanced models.

Because new mathematical operations—or other fundamental changes to neural-network calculations—often drive the model advances, adopting these models in software means porting the changes, too. Since research directions are hard to predict and advances are frequent, keeping a framework up to date and able to run the newest, best models requires a lot of work. For instance, while TensorFlow has more than 1,400 operations (TensorFlow, 2020e), TensorFlow Lite, which is deployed on more than four billions edge devices worldwide, supports only about 130 operations. Not all operations are worth supporting, however.

3 DESIGN PRINCIPLES

To address the challenges facing TinyML on embedded systems, we developed a set of developer principles to guide the design of the TF Micro framework to address the challenges mentioned previously in Section 2.

3.1 Minimize Feature Scope for Portability

We believe an embedded machine-learning (ML) framework should assume the model, input data, and output arrays are in memory, and it should only handle ML calculations based on those values. The design should exclude any other function, no matter how useful. In practice, this approach means the library should omit features such as loading models from a file system or accessing peripherals for inputs.

This strong design principle is crucial because many embedded platforms are missing basic features, such as memory management and library support (Section 2.1), that mainstream computing platforms take for granted. Supporting the myriad possibilities would make porting the ML framework across devices unwieldy.

Fortunately, ML models are functional, having clear inputs, outputs, and possibly some internal state but no external side effects. Running a model need not involve calls to peripherals or other operating-system functions. To remain efficient, we focus only on implementing those calculations.

3.2 Enable Vendor Contributions to Span Ecosystem

All embedded devices can benefit from high-performance kernels optimized for a given microprocessor. But no one team can easily support such kernels for the entire embedded market because of the ecosystem’s fragmentation (see Section 2.2). Worse, optimization approaches vary greatly depending on the target architecture.

The companies with the strongest motivation to deliver maximum performance on a set of devices are the ones that design and sell the underlying embedded microprocessors. Although developers at these companies are highly experienced at optimizing traditional numerical algorithms (e.g., digital signal processing) for their hardware, they often lack deep-learning experience. Therefore, evaluating whether optimization changes are detrimental to model accuracy and overall performance is difficult.

To improve the development experience for hardware vendors and application developers, we make sure optimizing the core library operations is easy. One goal is to ensure substantial technical support (tests and benchmarks) for developer modifications and to encourage submission to a library repository (details in Section 4).

3.3 Reuse TensorFlow Tools for Scalability

The TensorFlow training environment includes more than 1,400 operations, similar to other training frameworks (TensorFlow, 2020e). Most inference frameworks, however, explicitly support only a subset of these operations, making exports difficult. An exporter takes a trained model (such as a TensorFlow model) and generates a TensorFlow Lite model file (.tflite); after conversion, the model file can be deployed to a client device (e.g., a mobile or embedded system) and run locally using the TensorFlow Lite interpreter. Exporters receive a constant stream of new operations, most defined only by their implementation code. Because the operands lack clean semantic definitions beyond their implementations and unit tests, supporting these operations is difficult. Attempting to do so is like working with the elaborate CISC ISA without access to a basic data sheet.

Manually converting one or two models (and all the associated operations) to a new representation is easy. Users will want to convert a large space of potential models, however, and the task of understanding and changing model architectures to accommodate a framework’s requirements is difficult. Often, only after users have built and trained a model do they discover whether all of its operations are compatible with the target inference framework. Worse, many users employ high-level APIs, such as Keras (Chollet et al., 2015), which may hide low-level operations, complicating the task of removing dependence on operations. Also, researchers and product developers often split responsibilities,

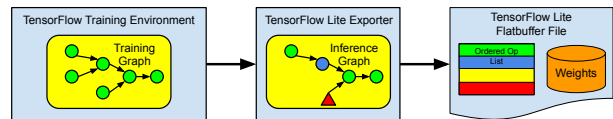


Figure 1. Model-export workflow.

with the former creating models and the latter deploying them. Since product developers are the ones who discover the export errors, they may lack the expertise or permission to retrain the model.

Model operators have no governing principles or a unified set of rules. Even if an inference framework supports an operation, particular data types may not, or the operation may exclude certain parameter ranges or may only serve in conjunction with other operations. This situation creates a barrier to providing error messages that guide developers.

Resource constraints also add many requirements to an exporter. Most training frameworks focus on floating-point calculations, since they are the most flexible numerical representation and are well optimized for desktop CPUs and GPUs. Fitting into small memories, however, makes eight-bit and other quantized representations valuable for embedded deployment. Some techniques can convert a model trained in floating point to a quantized representation (Krishnamoorthi, 2018), but they all increase exporter complexity. Some also require support during the training process, necessitating changes to the creation framework as well. Other optimizations are also expected during export, such as folding constant expressions into fixed values—even in complex cases like batch normalization (Zhang et al., 2017)—and removing dropout and similar operations that are only useful during training (Srivastava et al., 2014).

Because writing a robust model converter takes a tremendous amount of engineering work, we built atop the existing TensorFlow Lite tool chain, as Figure 1 shows. We exploited the strong integration with the TensorFlow training environment and extended it for deeply embedded machine-learning systems. For example, we reused the TensorFlow Lite reference kernels in TF Micro, thus giving users a harmonized environment for model development and execution.

3.4 Build System for Heterogeneous Support

Another crucial feature of an embedded inference framework is a flexible build environment. The build system must support the highly heterogeneous ecosystem and avoid falling captive to any one platform. Otherwise, developers would avoid adopting it due to the lack of portability and so would the hardware platform vendors.

In desktop and mobile systems, frameworks commonly provide precompiled libraries and other binaries as the main

software-delivery method. This approach is impractical in embedded platforms because they encompass too many different devices, operating systems, and tool-chain combinations to allow a balancing of modularity, size, and other constraints. Additionally, embedded developers must often make code changes to meet such constraints.

In response, we prioritize code that is easy to build using a wide variety of IDEs and tool chains. This approach means we avoid techniques that rely on build-system features that do not generalize across platforms. Examples of such features include setting custom include paths, compiling tools for the host processor, using custom binaries or shell scripts to produce code, and defining preprocessor macros on the command line.

Our principle is that we should be able to create source files and headers for a given platform, and users should then be able to drag and drop those files into their IDE or tool chain and compile them without any changes. We call it the “Bag of Files” principle. Anything more complex would prevent adoption by many platforms and developers.

4 IMPLEMENTATION

In this section, we discuss our implementation decisions and tradeoffs. We begin with a system overview (Figure 2) and then describe specific modules in detail.

4.1 System Overview

The first step in developing a TF Micro application is to create a live neural-network-model object in memory. To do so, the application developer produces an “operator resolver” object through the client API. The “OpResolver” API controls which operators link to the final binary, minimizing executable size.

The second step is to supply a contiguous memory array, called the “arena,” that holds intermediate results and other variables the interpreter needs. Doing so is necessary because we assume dynamic memory allocation, such as malloc or new, is unavailable.

The third step is to create an interpreter instance (Section 4.2), supplying it with the model, operator resolver, and arena as arguments. The interpreter allocates all required memory from the arena during the initialization phase. We intentionally avoid any allocations afterward to ensure heap fragmentation avoids causing errors for long-running applications. Operator implementations may need to allocate memory for use during the evaluation, so the preparation functions of each operator are called during this phase, allowing their memory requirements to be communicated to the interpreter. The application-supplied OpResolver maps the operator types listed in the serialized model to the im-

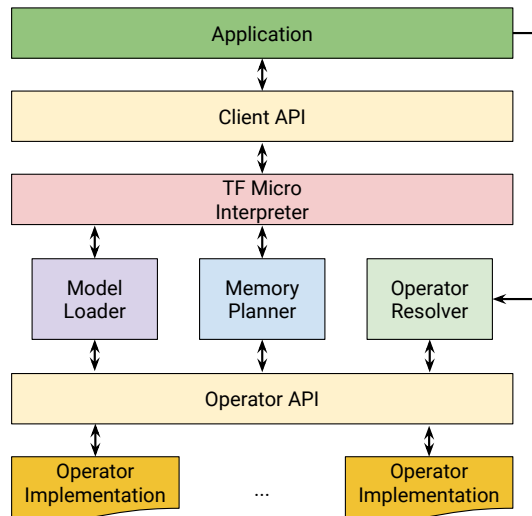


Figure 2. Implementation-module overview.

plementation functions.

A C API call handles all communication between the interpreter and operators to ensure operator implementations are modular and independent of the interpreter’s implementation. This approach eases replacement of operator implementations with optimized versions, and it also encourages reuse of other systems’ operator libraries (e.g., as part of a code-generation project).

The fourth step, after initialization, is model execution. The application retrieves pointers to the memory regions that represent the model inputs and populates them with values (often derived from sensors or other user-supplied data). Once the inputs are available, the application invokes the interpreter to perform the model calculations. This process involves iterating through the topologically sorted operations, using offsets calculated during memory planning to locate the inputs and outputs, and calling the evaluation function for each operation.

Finally, after it evaluates all the operations, the interpreter returns control to the application. Invocation is a simple blocking call, but an application can still perform one from a thread, and platform-specific operators can still split their work across processors. Once invocation finishes, the application can query the interpreter to determine the location of the arrays containing the model-calculation outputs and then use those outputs.

The framework omits any threading or multitasking support, since any such features would require less-portable code and operating-system dependencies. However, we support multitasking. The framework can run multiple models as long as they do not need to run concurrently with one another.

4.2 TF Micro Interpreter

TF Micro is an interpreter-based machine-learning framework. The interpreter loads a data structure that clearly defines a model. Although the execution code is static, the interpreter handles the model data at run time, and this data controls which operators to execute and where to draw the model parameters from.

We chose an interpreter on the basis of our experience deploying production models on embedded hardware. We see a need to easily update models in the field—a task that may be infeasible using code generation. Using an interpreter, however, sharing code across multiple models and applications is easier, as is maintaining the code, since it allows updates without re-exporting the model.

The alternative to an interpreter-based inference engine is to generate native code from a model during export using C or C++, baking operator function calls into fixed machine code. It can increase performance at the expense of portability, since the code would need recompilation for each target.

We incorporate some important code-generation features in our approach. For example, because our library is buildable from source files alone (Section 3.4), we achieve much of the compilation simplicity of generated code.

4.3 Model Loading

As mentioned, the interpreter loads a data structure that clearly defines a model. For this work, we used the TensorFlow Lite portable data schema (TensorFlow, 2020b). Reusing the export tools from TensorFlow Lite enabled us to import a wide variety of models at little engineering cost.

4.3.1 Model Serialization

TensorFlow Lite for smartphones and other mobile devices employs the FlatBuffer serialization format to hold models (TensorFlow, 2020a). The binary footprint of the accessor code is typically less than two kilobytes. It is a header-only library, making compilation easy, and it is memory efficient because the serialization protocol does not require unpacking to another representation.

The downside to this format is that its C++ header requires the platform compiler to support the C++11 specification. We had to work with several vendors to upgrade their tool chains to handle this version, but since we had implicitly chosen modern C++ by basing our framework on TensorFlow Lite, it has been a minor obstacle.

Another challenge of this format was that most of our target devices lacked file systems, but because it uses a memory-mapped representation, files are easy to convert into C source files containing data arrays. These files are compilable into the binary, to which the application can refer.

4.3.2 Model Representation

We also copied the TensorFlow Lite representation, the stored schema of data and values that represent the model. This schema was designed for mobile platforms with storage efficiency and fast access in mind, so it has many features that eased development for embedded platforms. For example, operations reside in a topologically sorted list rather than a directed-acyclic graph. Performing calculations is as simple as looping through the operation list in order, whereas a full graph representation would require preprocessing to satisfy the operations' input dependencies.

The biggest drawback of this representation is that it was designed to be portable from system to system, so it requires run-time processing to yield the information that inferencing requires. For example, it abstracts operator parameters from the arguments, which later pass to the functions that implement those operations. Thus, each operation requires a few code lines executed at run time to convert from the serialized representation to the structure in the underlying implementation. The code-size overhead is small, but it reduces the readability and compactness of the operator implementations.

Memory planning is a related issue. On mobile devices, TensorFlow Lite supports variable-size inputs, so all dependent operations may also vary in size. Planning the optimal memory layout of intermediate buffers for the calculations must therefore take place at run time when all buffer dimensions are known.

4.4 Memory Management

We are unable to assume the operating system can dynamically allocate memory. So the framework allocates and manages memory from a provided memory arena. During model preparation, the interpreter determines the lifetime and size of all buffers necessary to run the model. These buffers include run-time tensors, persistent memory to store metadata, and scratch memory to temporarily hold values while the model runs (Section 4.4.1). After accounting for all required buffers, the framework creates a memory plan that reuses nonpersistent buffers when possible while ensuring buffers are valid during their required lifetime (Section 4.4.2).

4.4.1 Persistent Memory and Scratchpads

We require applications to supply a fixed-size memory arena when they create the interpreter and to keep the arena intact throughout the interpreter's lifetime. Allocations with the same lifetime can treat this arena as a stack. If an allocation takes up too much space, we raise an application-level error.

To prevent memory errors from interrupting a long-running program, we ensure that allocations only occur during the interpreter's initialization phase. No allocation (through our

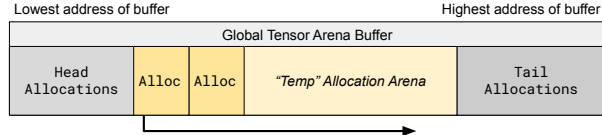


Figure 3. Two-stack allocation strategy.

mechanisms) is possible during model invocation.

This simplistic approach works well for initial prototyping, but it wastes memory because many allocations could overlap with others in time. One example is data structures that are only necessary during initialization, but because their lifetime is the same as the interpreter's, they continue to take up arena space. A model's evaluation phase also requires variables that need not persist from one invocation to another.

Hence, we modified the allocation scheme so that initialization- and evaluation-lifetime allocations reside in a separate stack relative to interpreter-lifetime objects. This feat uses a stack that increments from the lowest address for the function-lifetime objects ("Head" in Figure 3) and a stack that decrements from the arena's highest address for interpreter-lifetime allocations ("Tail" in Figure 3). When the two stack pointers cross, they indicate a lack of capacity.

The two-stack allocation strategy works well for both shared buffers and persistent buffers. But model preparation also holds allocation data that model inference no longer needs. Therefore, we used the space in between the two stacks as temporary allocations when a model is in memory planning. Any temporary data required during model inference resides in the persistent-stack allocation section.

Overall, our approach reduces the arena's required memory because the initialization allocations can be discarded after that function is done, and the memory is reusable for evaluation variables. This approach also enables advanced applications to reuse the arena's function-lifetime section in between evaluation calls.

4.4.2 Memory Planner

A more complex optimization opportunity involves the space required for intermediate calculations during model evaluation. An operator may write to one or more output buffers, and later operators may later read them as inputs. If the output is not exposed to the application as a model output, its contents need only remain until the last operation that needs them has finished. Its presence is also unnecessary until just before the operation that populates it executes. Memory reuse is possible by overlapping allocations that are unneeded during the same evaluation sections.

The memory allocations required over time can be visual-

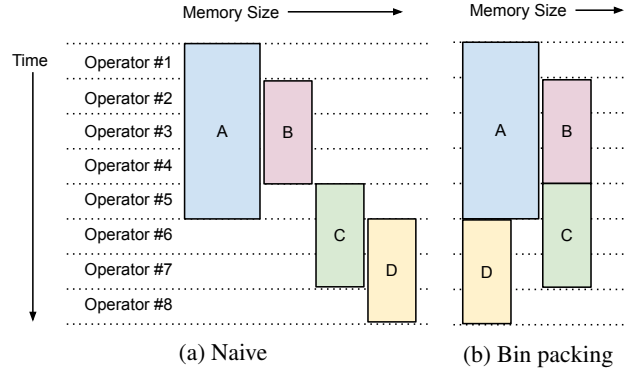


Figure 4. Intermediate allocation strategies.

ized using rectangles (Figure 4a), where one dimension is memory size and the other is the time during which each allocation must be preserved. The overall memory can be substantially reduced if some areas are reused or compacted together. Figure 4b shows a more optimal memory layout.

Memory compaction is an instance of bin packing (Martello, 1990). Calculating the perfect allocation strategy for arbitrary models without exhaustively trying all possibilities is an unsolved problem, but a first-fit decreasing algorithm (Garey et al., 1972) usually provides reasonable solutions.

In our case, this approach consists of gathering a list of all temporary allocations, including size and lifetime; sorting the list in descending order by size; and placing each allocation in the first sufficiently large gap, or at the end of the buffer if no such gap exists. We do not support dynamic shapes in the TF Micro framework, so we must know at initialization all the information necessary to perform this algorithm. The "Memory Planner" encapsulates this process (Figure 2); it allows us to minimize the arena portion devoted to intermediate tensors. Doing so offers a substantial memory-use reduction for many models.

Memory planning at run time incurs more overhead during model preparation than a preplanned memory-allocation strategy. This cost, however, comes with the benefit of model generality. TF Micro models simply list the operator and tensor requirements. At run time, we allocate and enable this capability for many model types.

Offline-planned tensor allocation is an alternative memory-planning feature of TF Micro. It allows a more compact memory plan, gives memory-plan ownership and control to the end user, imposes less overhead on the MCU during initialization, and enables more-efficient power options by allowing different memory banks to store certain memory areas. We allow the user to create a memory layout on a host before run time. The memory layout is stored as model FlatBuffer metadata and contains an array of fixed-memory arena offsets for an arbitrary number of variable tensors.

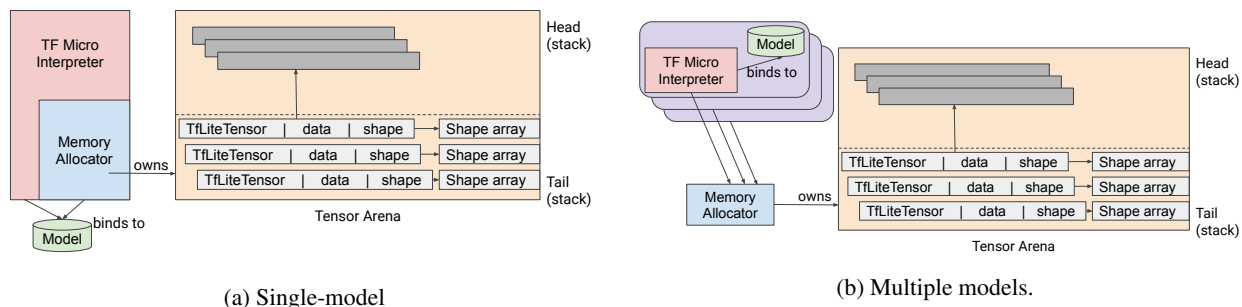


Figure 5. Memory-allocation strategy for a single model versus a multi-tenancy scenario. In TF Micro, there is a one-to-one binding between a model, an interpreter and the memory allocations made for the model (which may come from a shared memory arena).

4.5 Multitenancy

Embedded-system constraints can force application-model developers to create several specialized models instead of one large monolithic model. Hence, supporting multiple models on the same embedded system may be necessary.

If an application has multiple models that need not run simultaneously, TF Micro supports multitenancy with some memory-planner changes that are transparent to the developer. TF Micro supports memory-arena reuse by enabling the multiple model interpreters to allocate memory from a single arena. We allow interpreter-lifetime areas to stack on each other in the arena and reuse the function-lifetime section for model evaluation. The reusable (nonpersistent) part is set to the largest requirement, based on all models allocating in the arena. The nonreusable (persistent) allocations grow for each model—allocations are model specific.

4.6 Multi-threading

TF Micro is thread-safe as long as there is no state corresponding to the model that is kept outside the interpreter and the model’s memory allocation within the arena. The interpreter’s only variables are kept in the arena, and each interpreter instance is uniquely bound to a specific model. Therefore, TF Micro can safely support multiple interpreter instances running from different tasks or threads.

TF Micro can also run safely on multiple MCU cores. Since the only variables used by the interpreter are kept in the arena, this works well in practice. The executable code is shared, but the arenas ensure there are no threading issues.

4.7 Operator Support

Operators are the calculation units in neural-network graphs. They represent a sizable amount of computation, typically requiring many thousands or even millions of individual arithmetic operations (e.g., multiplies or additions). They are functional, with well-defined inputs, outputs, and state variables as well as no side effects beyond them.

Because the model execution’s latency, power consumption, and code size tend to be dominated by the implementations of these operations, they are typically specialized for particular platforms to take advantage of hardware characteristics. We attracted library optimizations from hardware vendors such as Arm, Cadence, Ceva, and Synopsys.

Well-defined operator boundaries mean it is possible to define an API that communicates the inputs and outputs but hides implementation details behind an abstraction. Several chip vendors have provided a library of neural network kernels designed to deliver maximum neural-network performance when running on their processors. For example, Arm has provided optimized CMSIS-NN libraries divided into several functions, each covering a category: convolution, activation, fully connected layer, pooling, softmax, and optimized basic math. TF Micro uses CMSIS-NN to deliver high performance as we demonstrate in Section 5.

4.8 Platform Specialization

TF Micro gives developers flexibility to modify the library code. Because operator implementations (kernels) often consume the most time when executing models, they are prominent targets for platform-specific optimization.

We wanted to make swapping in new implementations easy. To do so, we allow specialized versions of the C++ source code to override the default reference implementation. Each kernel has a reference implementation in a directory, but subfolders contain optimized versions for particular platforms (e.g., the Arm CMSIS-NN library).

As we explain in Section 4.9, the platform-specific source files replace the reference implementations during all build steps when targeting the named platform or library (e.g., using `TAGS="cmsis-nn"`). Each platform is given a unique tag. The tag is a command line argument to the build system that replaces the reference kernels during compilation. In a similar vein, library modifiers can swap or change the implementations incrementally with no changes to the build scripts and the overarching build system we put in place.

4.9 Build System

To address the embedded market’s fragmentation (Section 2.2), we needed our code to compile on many platforms. We therefore wrote the code to be highly portable, exhibiting few dependencies, but it was insufficient to give potential users a good experience on a particular device.

Most embedded developers employ a platform-specific IDE or tool chain that abstracts many details of building subcomponents and presents libraries as interface modules. Simply giving developers a folder hierarchy containing source-code files would still leave them with multiple steps before they could build and compile that code into a usable library.

Therefore, we chose a single `makefile` based build system to determine which files the library required, then generated the project files for the associated tool chains. The `makefile` held the source-file list, and we stored the platform-specific project files as templates that the project-generation process filled in with the source-file information. That process may also perform other postprocessing to convert the source files to a format suitable for the target tool chain.

Our platform-agnostic approach has enabled us to support a variety of tool chains with minimal engineering work, but it does have some drawbacks. We implemented the project generation through an ad hoc mixture of `makefile` scripts and Python. This strategy makes the process difficult to debug, maintain, and extend. Our intent is for future versions to keep the concept of a master source-file list that only the `makefile` holds, but then delegate the actual generation to better-structured Python in a more maintainable way.

5 SYSTEM EVALUATION

TF Micro has undergone testing and it has been deployed extensively with many processors based on the Arm Cortex-M architecture (Arm, 2020). It has been ported to other architectures including ESP32 (Espressif, 2020) and many digital signal processors (DSPs). The framework is also available as an Arduino library. It can generate projects for environments such as Mbed (ARM, 2020) as well. In this section, we use two representative platforms to assess and quantify TF Micro’s computational and memory overheads.

5.1 Experimental Setup

We selected two platforms on which to evaluate TF Micro (Table 1). First is the Sparkfun Edge, which has an Ambiq Apollo3 MCU. Apollo3 is powered by an Arm Cortex-M4 core and operates in burst mode at 96 MHz (Ambiq Micro, 2020). The second platform is an Xtensa HiFi Mini DSP, which is based on the Cadence Tensilica architecture (Cadence, 2020).

Our benchmarks are INT8 TensorFlow Lite models in a

serialized FlatBuffer format. We use the Visual Wake Words (VWW) person-detection model (Chowdhery et al., 2019), which represents a common microcontroller vision task of identifying whether a person appears in a given image. The model is trained and evaluated on images from the Microsoft COCO data set (Lin et al., 2014). It primarily stresses and measures the performance of convolutional operations.

Also, we use the Google Hotword model, which aids in detecting the key phrase “OK Google.” This model is designed to be small and fast enough to run constantly on a low-power DSP in smartphones and other devices with Google Assistant. Because it is proprietary, we use a version with scrambled weights and biases.

The benchmarks run multiple inputs through a single model, measuring the time to process each input and produce an inference output. The benchmark does not measure the time necessary to bring up the model and configure the run time, since the recurring inference cost dominates total CPU cycles on most long-running systems.

5.2 Benchmark Performance

We provide two sets of benchmark results. First are the baseline results from running the benchmarks on reference kernels, which are simple operator-kernel implementations designed for readability rather than performance. Second are results for optimized kernels compared with the reference kernels. The optimized versions employ high-performance ARM CMSIS-NN and Cadence libraries (Lai et al., 2018).

The results in Table 6 are for the CPU (Table 6a) and DSP (Table 6b). The total run time appears under the “Total Cycles” column, and the run time excluding the interpreter appears under the “Calculation Cycles” column. The difference between them is the interpreter overhead.

Comparing the reference kernel versions to the optimized kernel versions reveals considerable performance improvement. For example, between “VWW Reference” and “VWW Optimized,” the CMSIS-NN library offers more than a 4x speedup on the Cortex-M4 microcontroller. Optimization on the Xtensa HiFi Mini DSP offers a 7.7x speedup. For Hotword, the speeds are 25% and 50% better than the baseline reference model because less time goes to the kernel calculations and each inner loop accounts for less time with respect to the total run time of the benchmark model.

Platform	Processor	Clock	Flash	RAM
Sparkfun Edge (Ambiq Apollo3)	Arm CPU Cortex-M4	96 MHz	1 MB	0.38 MB
Tensilica HiFi	Xtensa DSP HiFi Mini	10 MHz	1 MB	1 MB

Table 1. Embedded-platform benchmarking.

Model	Total Cycles	Calculation Cycles	Interpreter Overhead
VWW Reference	18,990.8K	18,987.1K	< 0.1%
VWW Optimized	4,857.7K	4,852.9K	< 0.1%
Google Hotword Reference	45.1K	43.7K	3.3%
Google Hotword Optimized	36.4K	34.9K	4.1%

(a) Sparkfun Edge (Apollo3 Cortex-M4)

Model	Total Cycles	Calculation Cycles	Interpreter Overhead
VWW Reference	387,341.8K	387,330.6K	< 0.1%
VWW Optimized	49,952.3K	49,946.4K	< 0.1%
Google Hotword Reference	990.4K	987.4K	0.3%
Google Hotword Optimized	88.4K	84.6K	4.3%

(b) Xtensa HiFi Mini DSP

Figure 6. Performance results for TF Micro target platforms.

The “Interpreter Overhead” column in both Table 6a and Table 6b is insignificant compared with the total model run time on both the CPU and DSP. The overhead on the micro-controller CPU (Table 6a) is less than 0.1% for long-running models, such as VWW. In the case of short-running models such as Google Hotword, the overhead is still minimal at about 3% to 4%. The same general trend holds in Table 6b for non-CPU architectures like the Xtensa HiFi Mini DSP.

5.3 Memory Overhead

We assess TF Micro’s total memory usage. TF Micro’s memory usage includes the code size for the interpreter, memory allocator, memory planner, etc. plus any operators that are required by the model. Hence, the total memory usage varies greatly by the model. Large models and models with complex operators (e.g. VWW) consume more memory than their smaller counterparts like Google Hotword. In addition to VWW and Google Hotword, in this section, we added an even smaller reference convolution model containing just two convolution layers, a max-pooling layer, a dense layer, and an activation layer to emphasize the differences.

Overall, TF Micro applications have small footprint. Table 2 shows that for the convolutional and Google Hotword models, the memory consumed is at most 13 KB. For the larger VWW model, the framework consumes 26.5 KB.

To further analyze memory usage, recall that TF Micro allocates program memory into two main sections: persistent and nonpersistent. Table 2 reveals that depending on the model characteristics, one section can be larger than the other. The results show that we adjust to the needs of the different models while maintaining a small footprint.

5.4 Benchmarking and Profiling

TF Micro provides a set of benchmarks and profiling APIs (TensorFlow, 2020c) to compare hardware platforms and to let developers measure performance as well as identify opportunities for optimization. Benchmarks provide a consis-

Model	Persistent Memory	Nonpersistent Memory	Total Memory
Convolutional Reference	1.29 kB	7.75 kB	9.04 kB
VWW Reference	26.50 kB	55.30 kB	81.79 kB
Google Hotword Reference	12.12 kB	680 bytes	12.80 kB

Table 2. Memory consumption on Sparkfun Edge.

tent and fair way to measure hardware performance. MLPerf is adopting the benchmarks (Mattson et al., 2020; Reddi et al., 2020), and the tinyMLPerf benchmark suite imposes accuracy metrics for them (Banbury et al., 2020).

Although benchmarks measure performance, profiling is necessary to gain useful insights into model behavior. TF Micro has hooks for developers to instrument specific code sections (TensorFlow, 2020d). These hooks allow a TinyML application developer to measure overhead using a general-purpose interpreter rather than a custom neural-network engine for a specific model, and they can examine a model’s performance-critical paths. These features allow identification, profiling, and optimization of bottleneck operators.

6 RELATED WORK

Progress on frameworks targeting embedded devices is starting to rise. Besides frameworks, libraries that attempt to increase performance through optimized calls for MCU-based neural-network acceleration are in development as well. In this section, we discuss several related frameworks from both the industry as well as academia institutions.

Embedded machine learning is still developing with much headroom for research innovation, development, and deployment. Therefore, rather than make any head-on comparisons (as that would be not very meaningful at this early stage of evolution), we instead see this as an opportunity to identify

ongoing works that have the potential to mature and enable the broader ecosystem, much like our TF Micro effort.

ELL (Microsoft, 2020) The Embedded Learning Library (ELL) is an open-source library from Microsoft for embedded AI. ELL is a cross-compiler tool chain that enables users to run machine-learning models on resource constrained platforms, similar to the platforms that we have evaluated.

Graph Lowering (GLOW) (Rotem et al., 2018) is an open-source compiler that accelerates neural-network performance across a range of hardware platforms, both large and small. It initially targeted large machine-learning systems, but NXP recently extended it to focus on Arm Cortex-M MCUs and the Cadence Tensilica HiFi 4 DSPs. GLOW employs optimized kernels from vendor-supported libraries. Unlike TF Micro’s flexible interpreter-based solution, GLOW for MCUs is based on ahead-of-time compilation for both floating-point and quantized arithmetic.

STM32Cube.AI (STMicroelectronics, 2020) is the only other widely deployed production framework. It takes models from Keras, TensorFlow Lite, and others to generate code optimized for a range of STM32-series MCUs. It supports both FP32 and quantized models and comes with built-in optimizations to reduce model size. By comparison, TF Micro is more flexible, having been designed to serve a wide range of MCUs beyond the STMicroelectronics ecosystem.

TensorFlow-Native was an experimental Google system that compiled TensorFlow graphs into C++ code. The simplicity of the resulting code allowed porting of the system to many MCU and DSP targets. It lacked quantization support as well as platform-specific optimizations to achieve good performance. As we described previously in Section 3, we firmly believe that it is essential to leverage the existing infrastructure to enable broad adoption of the framework. Leveraging the existing toolchain is also essential to provide strong engineering support for product-level applications that run on many devices in the real-world.

TinyEngine (Lin et al., 2020) is an inference engine for MCUs. It is a code-generator-based compiler method that helps eliminate memory overhead. The authors claim it reduces memory usage by 2.7x and boosts the inference speed by 22% for their baseline. TF Micro, by contrast, uses an interpreter-based method, and as our experiments show, the interpreter adds insignificant overhead.

TVM (Chen et al., 2018) is an open-source deep-learning compiler for CPUs, GPUs, and machine-learning accelerators. It enables machine-learning engineers to optimize and run computations efficiently on any hardware back end. It has been ported to Arm’s Cortex-M7 and other MCUs.

uTensor (uTensor, 2020), a precursor to TF Micro, is a lightweight machine-learning inference framework specif-

ically designed for Arm. It consists of an offline tool that translates a TensorFlow model into C++ machine code, as well as a run time for execution management.

7 CONCLUSION

TF Micro enables the transfer of deep learning onto embedded systems, significantly broadening the reach of machine learning. TF Micro is a framework that has been specifically engineered to run machine learning effectively and efficiently on embedded devices with only a few kilobytes of memory. The framework fits in tens of kilobytes on microcontrollers and DSPs and can handle many basic models.

TF Micro’s fundamental contributions are the design decisions that address the challenges of embedded systems: hardware heterogeneity in the fragmented ecosystem, missing software features, and resource constraints. We support multiple embedded platforms based on the widely-deployed Arm Cortex-M series of microcontrollers, as well as other ISAs such as DSP cores from Tensilica. The framework does not require operating system support, any standard C or C++ libraries, or dynamic memory allocation – features that are commonly taken for granted in non-embedded system domains. This allows us to run bare-metal efficiently.

The methods and techniques presented here are a snapshot of the progress made so far. As embedded system capabilities grow, so will the framework. For example, we are in the process of developing an offline memory planner for more effective memory allocation and fine-grained user control, and investigating new approaches to support concurrent execution of ML models. In addition to minimizing memory consumption and improving performance, we are also looking into providing better support for vendor optimizations and build system support for development environments.

8 ACKNOWLEDGEMENTS

TF Micro is an open-source project and a community-based open-source project. As such, it rests on the work of many. We extend our gratitude to many individuals, teams, and organizations: Fredrik Knutsson and the CMSIS-NN team; Rod Crawford and Matthew Mattina from Arm; Raj Pawate from Cadence; Erich Plondke and Evgeni Gousef from Qualcomm; Jamie Campbell from Synopsys; Yair Siegel from Ceva; Sai Yelisetty from DSP Group; Zain Asgar from Stanford; Dan Situnayake from Edge Impulse; Neil Tan from the uTensor project; Sarah Sirajuddin, Rajat Monga, Jeff Dean, Andy Selle, Tim Davis, Megan Kacholia, Stella Laurenzo, Benoit Jacob, Dmitry Kalenichenko, Andrew Howard, Aakanksha Chowdhery, and Lawrence Chan from Google; and Radhika Ghosal, Sabrina Neuman, Mark Mazumder, and Colby Banbury from Harvard University.

REFERENCES

- Ambiq Micro. *Apollo 3 Blue Datasheet*, 2020. URL https://cdn.sparkfun.com/assets/learn_tutorials/9/0/9/Apollo3_Blue_MCU_Data_Sheet_v0.9-1.pdf.
- ARM. *Arm Enables Custom Instructions for embedded CPUs*, 2019. URL <https://www.arm.com/company/news/2019/10/arm-enables-custom-instructions-for-embedded-cpus>.
- ARM. *Mbed*, 2020. URL <https://os.mbed.com>.
- Arm. *Arm Cortex M*, 2020. URL <https://developer.arm.com/ip-products/processors/cortex-m>.
- Banbury, C. R., Reddi, V. J., Lam, M., Fu, W., Fazal, A., Holleman, J., Huang, X., Hurtado, R., Kanter, D., Lokhmotov, A., et al. Benchmarking tinyml systems: Challenges and direction. *arXiv preprint arXiv:2003.04821*, 2020.
- Cadence. *Tensilica Hi-Fi DSP Family*, 2020. URL https://ip.cadence.com/uploads/928/TIP_PB_HiFi_DSP_FINAL-pdf.
- Chavarriaga, R., Sagha, H., Calatroni, A., Digumarti, S. T., Tröster, G., Millán, J. d. R., and Roggen, D. The opportunity challenge: A benchmark database for on-body sensor-based activity recognition. *Pattern Recognition Letters*, 34(15):2033–2042, 2013.
- Chen, G., Parada, C., and Heigold, G. Small-footprint keyword spotting using deep neural networks. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4087–4091. IEEE, 2014.
- Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pp. 578–594, 2018.
- Chollet, F. et al. Keras, 2015. URL <https://keras.io/>.
- Chowdhery, A., Warden, P., Shlens, J., Howard, A., and Rhodes, R. Visual wake words dataset. *arXiv preprint arXiv:1906.05721*, 2019.
- Espressif. *Espressif ESP32*, 2020. URL <https://www.espressif.com/en/products/socs/esp32>.
- Garey, M. R., Graham, R. L., and Ullman, J. D. Worst-case analysis of memory allocation algorithms. In *Proceedings of the fourth annual ACM symposium on Theory of computing*, pp. 143–150, 1972.
- Goebel, K. et al. *NASA PCoE Datasets*, 2020. URL <https://ti.arc.nasa.gov/tech/dash/groups/pcoe/prognostic-data-repository/>.
- Gruenstein, A., Alvarez, R., Thornton, C., and Ghodrati, M. A cascade architecture for keyword spotting on mobile devices. *arXiv preprint arXiv:1712.03603*, 2017.
- IC Insights. *MCUs Expected to Make Modest Comeback after 2020 Drop*, 2020. URL <https://www.icinsights.com/news/bulletins/MCUs-Expected-To-Make-Modest-Comeback-After-2020-Drop--/>.
- Intel. Intel-64 and ia-32 architectures software developer’s manual. *Volume 3A: System Programming Guide, Part 1* (64), 2013.
- Koizumi, Y., Saito, S., Uematsu, H., Harada, N., and Imoto, K. ToyADMOS: A dataset of miniature-machine operating sounds for anomalous sound detection. In *Proceedings of IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, pp. 308–312, November 2019. URL <https://ieeexplore.ieee.org/document/8937164>.
- Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.
- Kumar, A., Goyal, S., and Varma, M. Resource-efficient machine learning in 2 kb ram for the internet of things. In *International Conference on Machine Learning*, pp. 1935–1944, 2017.
- Lai, L., Suda, N., and Chandra, V. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M cpus. *arXiv preprint arXiv:1801.06601*, 2018.
- Lin, J., Chen, W.-M., Lin, Y., Cohn, J., Gan, C., and Han, S. Mxnet: Tiny deep learning on IoT devices. *arXiv preprint arXiv:2007.10319*, 2020.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft COCO: Common objects in context. In *European conference on computer vision*, pp. 740–755. Springer, 2014.
- Martello, S. Chapter 8: Bin packing, knapsack problems: algorithms and computer implementations. *Wiley-Interscience series in discrete mathematics and optimization*, 1990.

- Mattson, P., Reddi, V. J., Cheng, C., Coleman, C., Diamos, G., Kanter, D., Micikevicius, P., Patterson, D., Schmuelling, G., Tang, H., et al. MLPerf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020.
- Microsoft. *Embedded Learning Library*, 2020. URL <https://microsoft.github.io/ELL/>.
- Reddi, V. J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., Wu, C.-J., Anderson, B., Breughe, M., Charlebois, M., Chou, W., et al. MLPerf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446–459. IEEE, 2020.
- Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhavarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., et al. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*, 2018.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- STMicroelectronics. *STM32Cube.AI*, 2020. URL https://www.st.com/content/st_com/en/stm32-ann.html.
- Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., and Beghi, A. Machine learning for predictive maintenance: A multiple classifier approach. *IEEE Transactions on Industrial Informatics*, 11(3):812–820, 2014.
- TensorFlow. *TensorFlow Lite FlatBuffer Model*, 2020a. URL https://www.tensorflow.org/lite/api_docs/cc/class/tflite/flat-buffer-model.
- TensorFlow. *TensorFlow Lite Guide*, 2020b. URL <https://www.tensorflow.org/lite/guide>.
- TensorFlow. *Tensorflow Lite Micro Benchmarks*, 2020c. URL <https://github.com/tensorflow/tensorflow/tree/master/tensorflow/lite/micro/benchmarks>.
- TensorFlow. *Tensorflow Lite Micro Profiler*, 2020d. URL https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/micro/micro_profiler.cc.
- TensorFlow. *TensorFlow Core Ops*, 2020e. URL <https://github.com/tensorflow/tensorflow/blob/master/tensorflow/core/ops/ops.pbtxt>.
- uTensor. *uTensor*, 2020. URL <https://github.com/uTensor/uTensor>.
- Waterman, A. and Asanovic, K. The risc-v instruction set manual, volume i: Unprivileged isa document, version 20190608-baseratified. *RISC-V Foundation, Tech. Rep*, 2019.
- Wu, X., Lee, I., Dong, Q., Yang, K., Kim, D., Wang, J., Peng, Y., Zhang, Y., Saliganc, M., Yasuda, M., et al. A 0.04 mm 3 16nw wireless and batteryless sensor system with integrated cortex-m0+ processor and optical communication for cellular temperature measurement. In *2018 IEEE Symposium on VLSI Circuits*, pp. 191–192. IEEE, 2018.
- Zhang, M. and Sawchuk, A. A. Usc-had: a daily activity dataset for ubiquitous activity recognition using wearable sensors. In *Proceedings of the 2012 ACM Conference on Ubiquitous Computing*, pp. 1036–1043, 2012.
- Zhang, Y., Suda, N., Lai, L., and Chandra, V. Hello edge: Keyword spotting on microcontrollers. *arXiv preprint arXiv:1711.07128*, 2017.